

~\OneDrive\Desktop\artificial neural networks\lab3\es2\iris-code.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Load the Iris dataset
5 # iris-data.csv contains 150 samples with 4 features each
6 # iris-labels.csv contains class labels (0=Setosa, 1=Versicolor, 2=Virginica)
7 data = np.loadtxt('iris-data.csv', delimiter=',')
8 # Load labels as float first (they're in scientific notation), then convert to int
9 labels = np.loadtxt('iris-labels.csv', delimiter=',').astype(int)
10
11 print(f"Dataset loaded: {data.shape[0]} samples, {data.shape[1]} features")
12 print(f"Classes: {np.unique(labels)}")
13
14 # Standardize the data by dividing by the maximum value in the dataset
15 max_value = np.max(data)
16 data_normalized = data / max_value
17 print(f>Data normalized by max value: {max_value:.2f}")
18
19 # SOM Parameters
20 n_features = 4 # input dimension (4 features per iris sample)
21 grid_size = 40 # 40x40 output array
22 n_epochs = 10
23 batch_size = 1 # stochastic learning
24 n_samples = len(data_normalized)
25
26 # Initial hyperparameters
27 eta0 = 0.1 # initial learning rate
28 d_eta = 0.01 # learning rate decay
29 sigma0 = 10.0 # initial neighborhood width
30 d_sigma = 0.05 # neighborhood width decay
31
32 print(f"\nSOM Configuration:")
33 print(f"  Grid size: {grid_size}x{grid_size}")
34 print(f"  Learning rate:  $\eta_0$ ={eta0}, decay={d_eta}")
35 print(f"  Neighborhood width:  $\sigma_0$ ={sigma0}, decay={d_sigma}")
36
37 # Initialize weights randomly in [0, 1] using uniform distribution
38 np.random.seed(42) # for reproducibility
39 weights = np.random.uniform(0, 1, (grid_size, grid_size, n_features))
40
41 # Create position matrix for neurons in the output array
42 # Each neuron at grid position (i,j) has coordinates r_i = [i, j]
43 positions = np.zeros((grid_size, grid_size, 2))
44 for i in range(grid_size):
45     for j in range(grid_size):
46         positions[i, j] = [i, j]
47
48
49 def find_winning_neuron(x, w):
50     """
51     Find the Best Matching Unit (BMU) - the winning neuron i_0
```

```
52     This is the neuron whose weight vector is closest to input x
53
54     Args:
55         x: input pattern (n_features,)
56         w: weight array (grid_size, grid_size, n_features)
57
58     Returns:
59         tuple: (i, j) position of winning neuron
60     """
61     # Calculate Euclidean distance between input and all weight vectors
62     distances = np.sum((w - x)**2, axis=2)
63     # Find position of minimum distance
64     winner_idx = np.unravel_index(np.argmin(distances), distances.shape)
65     return winner_idx
66
67
68 def neighborhood_function(winner_pos, sigma):
69     """
70     Gaussian neighborhood function h(i, i_0) from Eq. 10.18
71     
$$h(i, i_0) = \exp(-||r_i - r_{i_0}||^2 / (2\sigma^2))$$

72
73     Args:
74         winner_pos: position [i, j] of winning neuron
75         sigma: width of neighborhood function
76
77     Returns:
78         array: neighborhood values for all neurons (grid_size, grid_size)
79     """
80     # Calculate squared distances from winner to all neurons
81     distances_sq = np.sum((positions - winner_pos)**2, axis=2)
82     # Apply Gaussian function (Eq. 10.18)
83     h = np.exp(-distances_sq / (2 * sigma**2))
84     return h
85
86
87 # Store winning neuron positions BEFORE training (with random weights)
88 print("\nFinding winning neurons with initial random weights...")
89 winners_before = []
90 for i in range(n_samples):
91     winner = find_winning_neuron(data_normalized[i], weights)
92     winners_before.append(winner)
93
94 # Training loop - implement Kohonen's learning rule (Eq. 10.17)
95 print("\nStarting training...")
96 for epoch in range(n_epochs):
97     # Update learning rate and neighborhood width with exponential decay
98     eta = eta0 * np.exp(-d_eta * epoch)
99     sigma = sigma0 * np.exp(-d_sigma * epoch)
100
101     # Shuffle data indices for stochastic gradient descent (batch_size = 1)
102     indices = np.random.permutation(n_samples)
103
104     # Process each sample
105     for idx in indices:
```

```
106         x = data_normalized[idx]
107
108         # Find winning neuron i_0 (BMU)
109         winner_pos = find_winning_neuron(x, weights)
110
111         # Calculate neighborhood function h(i, i_0) for all neurons
112         h = neighborhood_function(positions[winner_pos[0], winner_pos[1]], sigma)
113
114         # Update weights using Kohonen's rule (Eq. 10.17):
115         #  $\delta w_i = \eta * h(i, i_0) * (x - w_i)$ 
116         for i in range(grid_size):
117             for j in range(grid_size):
118                 weights[i, j] += eta * h[i, j] * (x - weights[i, j])
119
120         print(f"Epoch {epoch + 1}/{n_epochs} completed -  $\eta$ ={eta:.4f},  $\sigma$ ={sigma:.4f}")
121
122     # Store winning neuron positions AFTER training
123     print("\nFinding winning neurons after training...")
124     winners_after = []
125     for i in range(n_samples):
126         winner = find_winning_neuron(data_normalized[i], weights)
127         winners_after.append(winner)
128
129     # Create figure with two panels as required
130     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))
131
132     # Define colors for the three Iris species
133     colors = ['red', 'blue', 'green']
134     class_names = ['Iris-setosa', 'Iris-versicolor', 'Iris-virginica']
135
136     # Panel 1: BEFORE training (random weights)
137     ax1.set_title('Before Training: Random Weights', fontsize=15, fontweight='bold', pad=15)
138     ax1.set_xlabel('Neuron Column Position', fontsize=12)
139     ax1.set_ylabel('Neuron Row Position', fontsize=12)
140     ax1.set_xlim(-1, grid_size)
141     ax1.set_ylim(-1, grid_size)
142     ax1.grid(True, alpha=0.3, linestyle='--')
143     ax1.set_aspect('equal')
144
145     # Plot winning neuron locations colored by class
146     for i in range(n_samples):
147         pos = winners_before[i]
148         class_label = labels[i]
149         ax1.scatter(pos[1], pos[0], c=colors[class_label], s=60, alpha=0.6,
150                    edgecolors='black', linewidth=0.5)
151
152     # Panel 2: AFTER training (organized clusters)
153     ax2.set_title('After Training: Self-Organized Clusters', fontsize=15, fontweight='bold',
154                  pad=15)
155     ax2.set_xlabel('Neuron Column Position', fontsize=12)
156     ax2.set_ylabel('Neuron Row Position', fontsize=12)
157     ax2.set_xlim(-1, grid_size)
158     ax2.set_ylim(-1, grid_size)
159     ax2.grid(True, alpha=0.3, linestyle='--')
```

```
159 ax2.set_aspect('equal')
160
161 # Plot winning neuron locations colored by class
162 for i in range(n_samples):
163     pos = winners_after[i]
164     class_label = labels[i]
165     ax2.scatter(pos[1], pos[0], c=colors[class_label], s=60, alpha=0.6,
166                edgecolors='black', linewidth=0.5)
167
168 # Create legend with the three Iris species
169 legend_elements = [plt.Line2D([0], [0], marker='o', color='w',
170                               markerfacecolor=colors[i], markersize=11,
171                               markeredgecolor='black', markeredgewidth=0.5,
172                               label=class_names[i]) for i in range(3)]
173 fig.legend(handles=legend_elements, loc='lower center', bbox_to_anchor=(0.5, -0.02),
174            ncol=3, fontsize=12, frameon=True, fancybox=True, shadow=True)
175
176 plt.tight_layout(rect=[0, 0.03, 1, 1])
177 plt.savefig('som_iris_results.png', dpi=300, bbox_inches='tight')
178 plt.show()
179
180 print("\n" + "="*60)
181 print("Training completed successfully!")
182 print("="*60)
183 print(f"Total samples processed: {n_samples}")
184 print(f"Iris species: {class_names}")
185 print(f"Figure saved as: som_iris_results.png")
186 print("\nResults:")
187 print(" - Left panel: shows random distribution before training")
188 print(" - Right panel: shows organized clusters after training")
189 print(" - Each color represents one Iris species")
190 print(" - Similar inputs activate nearby neurons (topographic mapping)")
```